

Machine-Learning-Based Predictive Maintenance of Industrial Assets: A Case Study on the NASA C-MAPSS Turbofan Engine Degradation Dataset

Moksh Mehta
Brown University
moksh_mehta@brown.edu

Independent study under Prof. Serdar Kadioglu
Spring 2026

Abstract

Industrial condition monitoring devices — vibration sensors, thermocouples, pressure transducers and the like — produce dense multivariate time series that can in principle be used to predict machine failure before it happens, replacing scheduled maintenance with condition-based intervention. This independent study develops an end-to-end machine-learning pipeline for that task and benchmarks it on the publicly released NASA C-MAPSS turbofan engine degradation dataset (Saxena and Goebel, 2008), the canonical academic benchmark for remaining-useful-life (RUL) prediction. We document a complete data preparation pipeline (regime detection, regime-aware standardisation, rolling statistical features, piecewise-linear RUL targets); train and evaluate five tabular baselines (mean predictor, ridge regression, random forest, XGBoost, LightGBM) and a two-layer LSTM on all four CMAPSS subsets; and ship the best model behind a Flask REST API that returns a numeric RUL prediction together with a categorical maintenance recommendation. On the held-out CMAPSS test sets our best models achieve RMSE of **15.27**, **14.74**, **14.47** and **15.50** cycles on FD001–FD004 respectively, within striking distance of the state-of-the-art deep models reported in the prognostics literature. Along the way we identified and document a subtle data-leakage failure mode — a normalised cycle-counter feature that perfectly correlates with RUL on training engines (which all ran to failure) but is uninformative on test engines (which are truncated mid-life) — that inflated tree-model RMSE from 17 to 85 cycles. This makes a useful cautionary example for industrial deployments. All code and results are released in an accompanying repository.

1 Introduction

Industrial assets such as jet engines, pumps, bearings and gearboxes are typically maintained on one of three regimes: *run-to-failure* (cheapest in capex, most expensive in unplanned downtime), *scheduled/preventive* (replacing parts at fixed intervals, regardless of their actual state), or *condition-based / predictive* (intervening only when the data indicate the asset is approaching failure) (Lei et al., 2018). The third regime, predictive maintenance (PdM), promises lower total cost of ownership but requires a model that can map dense streams of sensor data to an actionable estimate of the asset’s future state.

The supervisor and student behind this independent study originally planned to use proprietary condition-monitoring data from SPM Instrument and its clients — vibration and frequency-domain

measurements, derived envelope statistics, paired with process variables such as temperature and throughput. Late in the planning phase it became clear that the proprietary data would not be available on the project’s timeline. Rather than pause the study, we redirected it onto the most widely used public benchmark for the same prediction task: the NASA C-MAPSS Turbofan Engine Degradation Simulation Dataset (Saxena and Goebel, 2008). The dataset has been used in hundreds of academic papers since 2008 — enough that we have a clear sense of what “good” looks like on each of its four subsets — and exposes exactly the same modelling questions as bearing-vibration data would have done: how to handle multiple operating regimes, how to choose a sensible regression target when the asset is healthy, how to evaluate fairly given asymmetric costs of early vs. late prediction, and how to turn a regression model into an operational decision.

Contributions. This report contributes:

1. A reusable, documented Python pipeline (data loading, regime-aware feature engineering, sequence builder, model training, REST API) released as a self-contained repository.
2. Empirical benchmarks of five classical regressors and an LSTM on all four CMAPSS subsets, reported on RMSE, MAE and the asymmetric PHM08 score.
3. A case study of one specific feature-leakage bug that, if shipped, would silently cause a deployed model to underestimate RUL by 50 cycles — and would not be caught by ordinary hold-out validation on training engines.
4. A simple deployment prototype (Flask REST API and a Streamlit dashboard) that maps numerical RUL predictions onto categorical maintenance actions.

The remainder of the paper is organised as follows. Section 2 states the problem and the evaluation metrics. Section 3 describes the dataset and our exploratory analysis. Section 4 details the feature pipeline and the models. Section 5 reports test-set performance and discusses the leakage pitfall. Section 6 describes the deployment prototype. Section 7 discusses limitations and what would change if the original proprietary data were used. Section 8 concludes.

2 Problem statement and evaluation

2.1 Definitions

For each asset (a turbofan engine, in our case) we observe a multivariate time series $\{x_t\}_{t=1}^T$ where each $x_t \in \mathbb{R}^d$ collects the values of d sensor channels at operational cycle t . In the training data the asset is observed all the way to failure, so $T = T_{\text{fail}}$. At any cycle $t \leq T_{\text{fail}}$ the asset’s *remaining useful life* (RUL) is the integer-valued quantity

$$\text{RUL}(t) = T_{\text{fail}} - t. \tag{1}$$

The supervised task is: given the history $x_{1:t}$ up to some cycle t , predict $\widehat{\text{RUL}}(t)$.

2.2 Evaluation metrics

Three metrics are standard in the CMAPSS literature.

RMSE. The root-mean-square error between predicted and true RUL on the test set. Symmetric and easy to interpret in cycles.

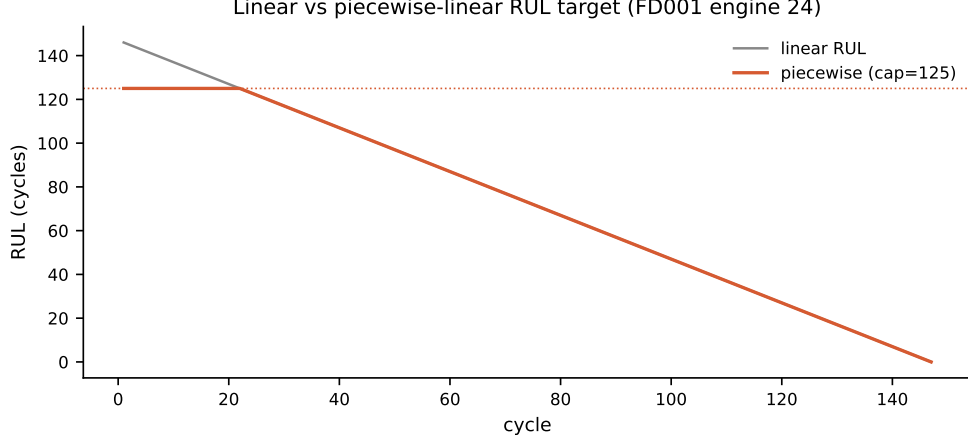


Figure 1: Linear vs. piecewise-linear RUL target for one FD001 engine. Capping at $R_{\max} = 125$ converts the early life of the engine into a single “healthy” value the model is not penalised for missing, focusing learning on the degradation phase.

MAE. The mean absolute error. Less sensitive than RMSE to the few engines whose dynamics are far from the rest of the fleet.

PHM08 score (Saxena and Goebel, 2008). The asymmetric scoring function introduced for the 2008 PHM Society competition,

$$S = \sum_{i=1}^N s_i, \quad s_i = \begin{cases} \exp(-d_i/13) - 1 & d_i < 0 \\ \exp(d_i/10) - 1 & d_i \geq 0 \end{cases}, \quad d_i = \widehat{\text{RUL}}_i - \text{RUL}_i. \quad (2)$$

The function is convex in d_i and the right-hand exponent (10) is smaller than the left-hand exponent (13), so a prediction that *over-estimates* RUL by 30 cycles incurs a much larger penalty than one that under-estimates it by the same amount. This codifies the operational fact that *a late prediction is worse than an early one*: an engineer who is told the engine has 50 cycles left and acts on that estimate suffers a catastrophic failure if the truth was 20.

2.3 The piecewise-linear RUL target

A pure linear RUL target (Eq. 1) is asking the model to do something physically impossible early in the engine’s life: distinguish a brand-new engine that will run for 300 cycles from one that will run for 250. Following Heimes (2008) we cap the training target at an upper bound (we use 125 cycles, the standard choice in subsequent literature) before fitting:

$$\text{RUL}^\dagger(t) = \min(\text{RUL}(t), R_{\max}), \quad R_{\max} = 125. \quad (3)$$

The intuition is shown in Figure 1: in the early, healthy phase the model is only asked to predict “ ≥ 125 ”; the useful signal-extraction problem is concentrated in the degradation phase.

3 Dataset and exploratory analysis

3.1 The C-MAPSS dataset

C-MAPSS (Frederick et al., 2007) (“Commercial Modular Aero-Propulsion System Simulation”) is a high-fidelity physics-based simulator of a large commercial turbofan engine, developed at NASA

Table 1: Summary of the four CMAPSS subsets. “Train cycles” is the total number of (engine, cycle) records available for supervised training; “mean RUL” is the average true RUL of the test-set engines at their last recorded cycle.

Subset	Op. conditions	Fault modes	Engines		Train cycles	Mean life (cycles)	
			train	test		train	test
FD001	1 (sea level)	1 (HPC)	100	100	20 631	206.3	131.0
FD002	6	1 (HPC)	260	259	53 759	206.8	131.2
FD003	1 (sea level)	2 (HPC + Fan)	100	100	24 720	247.2	166.0
FD004	6	2 (HPC + Fan)	249	248	61 249	246.0	166.2

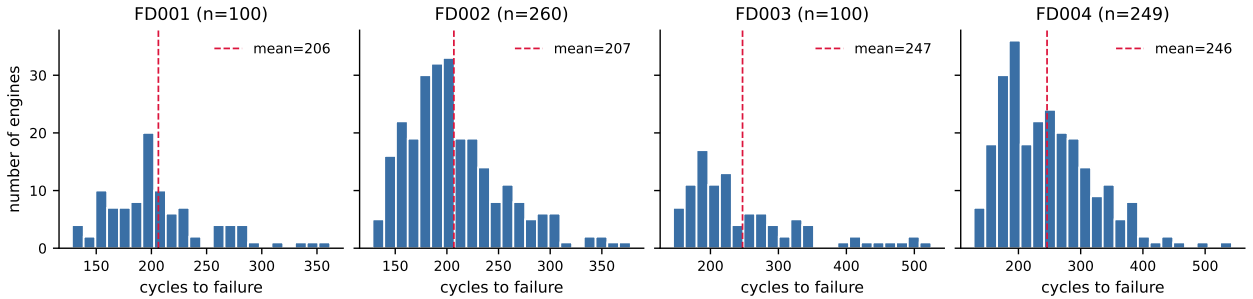


Figure 2: Distribution of run-to-failure lengths in the training set of each subset. Dashed red line is the mean.

Glenn Research Center. Saxena and Goebel (2008) used it to generate run-to-failure trajectories for a fleet of nominally identical engines, each with a small random degree of initial wear and manufacturing variation, and released four subsets of progressively increasing difficulty (Table 1).

Each row of every file records the three operational settings (altitude proxy, Mach proxy, throttle resolver angle) and 21 sensor measurements (temperatures, pressures, fan speeds, fuel flow, bleed enthalpy) for a single operational cycle of one engine.

3.2 Engine lifetimes

Figure 2 shows the distribution of engine lifetimes (cycles to failure) in the training sets. The mean is around 200–250 cycles but the right tail extends to 500+ cycles. Practically this means a model trained naively to predict RUL has to span a target range of 500 cycles even though most of the variance lives in the last 100, motivating the piecewise-linear target of Eq. 3.

3.3 Operating conditions

The simplest subset, FD001, is recorded entirely at one operating condition (sea-level cruise); the operating-setting columns are essentially constant. The more realistic FD002 and FD004 sweep through six discrete conditions corresponding to different altitude/Mach/throttle combinations. Figure 3a plots `op_setting_1` vs. `op_setting_2` for FD001 (one point cloud) and FD002 (six clusters). K -means with $k = 6$ trivially recovers the six clusters in FD002 and FD004 (Figure 3b), and we use this clustering downstream for regime-aware normalisation.

3.4 Sensor signals

Of the 21 raw sensor channels, seven (`s1`, `s5`, `s6`, `s10`, `s16`, `s18`, `s19`) carry essentially no variance in FD001 and FD003 — they are recorded faithfully by C-MAPSS but the underlying physical

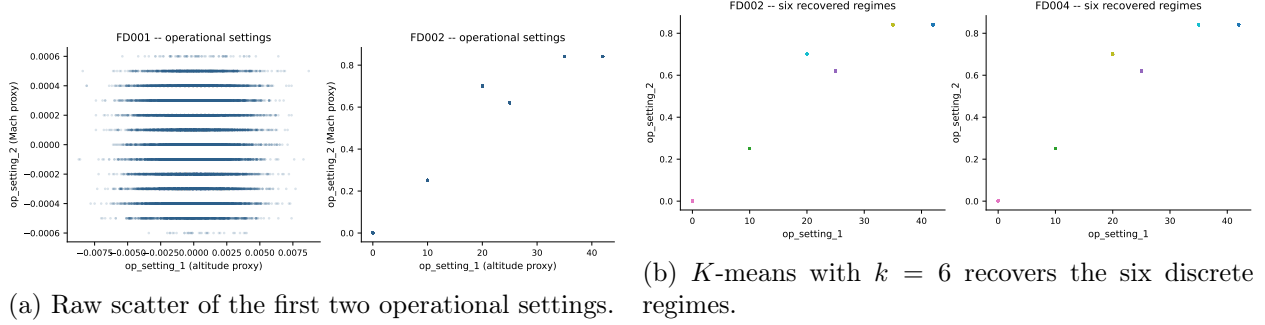


Figure 3: Operational regimes in CMAPSS. The multi-condition subsets FD002/FD004 contain exactly six well-separated flight regimes.

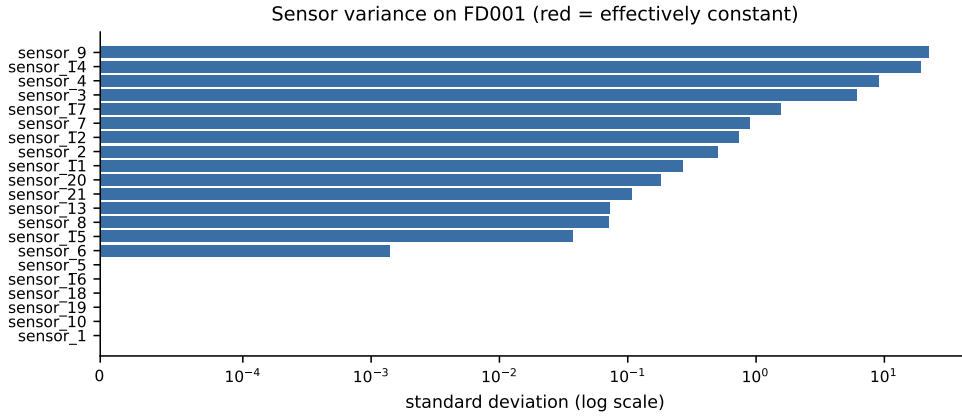


Figure 4: Per-channel standard deviation in FD001 (log scale). Seven channels (red) are effectively constant and are dropped from the feature set.

quantities are simply constant under sea-level cruise. Figure 4 plots the per-channel standard deviation in FD001 on a symmetric-log scale; we drop these channels from all subsequent modelling.

The remaining sensors exhibit a clear degradation signature when plotted across an engine’s lifetime (Figure 5): values drift monotonically (some upward, some downward) as the engine ages, with stochastic cycle-to-cycle variation around the trend.

Per-sensor correlation with RUL. Figure 6 ranks each sensor by its Pearson correlation with the (uncapped) RUL on each subset. On single-condition data (FD001, FD003) about ten sensors are visibly correlated ($|r| > 0.3$); on multi-condition data (FD002, FD004) the raw correlations are uniformly weak because the operating-condition swings dominate the cycle-to-cycle variation. This is the key motivation for regime-aware normalisation, discussed next.

4 Methods

4.1 Regime-aware normalisation

For single-condition subsets we simply z-score each (informative) sensor channel using the global mean and standard deviation computed on the training set. For the six-condition subsets we first cluster all training rows with K -means on the three operating settings ($k = 6$), fit a separate **StandardScaler** per cluster, and at inference time route each new row to its cluster’s scaler. This

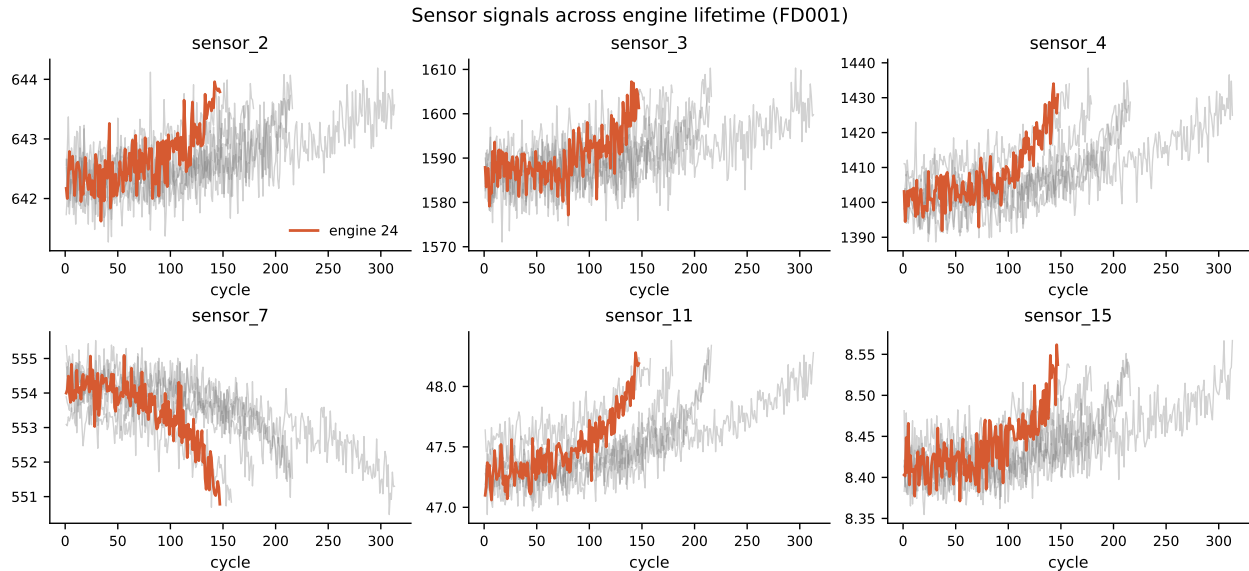


Figure 5: Six representative sensor traces across the lifetime of one FD001 engine (red) and several others (grey). The downward and upward drifts visible in the final 100–150 cycles of every trace are the degradation signature we are trying to predict.

standard trick (Ramasso and Saxena, 2014) removes the dominant operating-condition variance and exposes the underlying degradation signal (Figure 7).

4.2 Hand-engineered features for tabular models

On top of the standardised sensor values we compute, per engine and per window $w \in \{5, 20\}$:

- rolling mean $\bar{x}_w(t)$,
- rolling standard deviation $\sigma_w(t)$,
- rolling OLS slope $\hat{\beta}_w(t)$ of the channel against time, computed in closed form from cumulative sums to keep the cost $\mathcal{O}(N)$ rather than $\mathcal{O}(Nw)$.

This expands the per-cycle feature vector from 14 standardised sensors to $14 \times 7 = 98$ tabular features. We deliberately do *not* include the trivially-tempting feature `cycle_norm = cycle / max(cycle)`; the next section explains why this single line of code, easy to write and very hard to debug, broke our first tree models.

4.3 Sequence dataset for the recurrent model

For the LSTM we use a sliding window of length $L = 30$ cycles. For each training cycle t we extract $X_t = (x_{t-29}, \dots, x_t)$ as one example (front-padded with zeros for $t < 30$) with target $y_t = \min(\text{RUL}(t), 125)$. For each test engine we use only the final L cycles of the recorded run, paired with the supplied ground-truth final-cycle RUL. Sequences contain only the 14 standardised informative sensors (plus the regime label for FD002/FD004), without the hand-engineered rolling statistics, on the principle that the LSTM should learn whatever temporal aggregation it needs internally.

4.4 Models

We train and evaluate the following six models per subset:

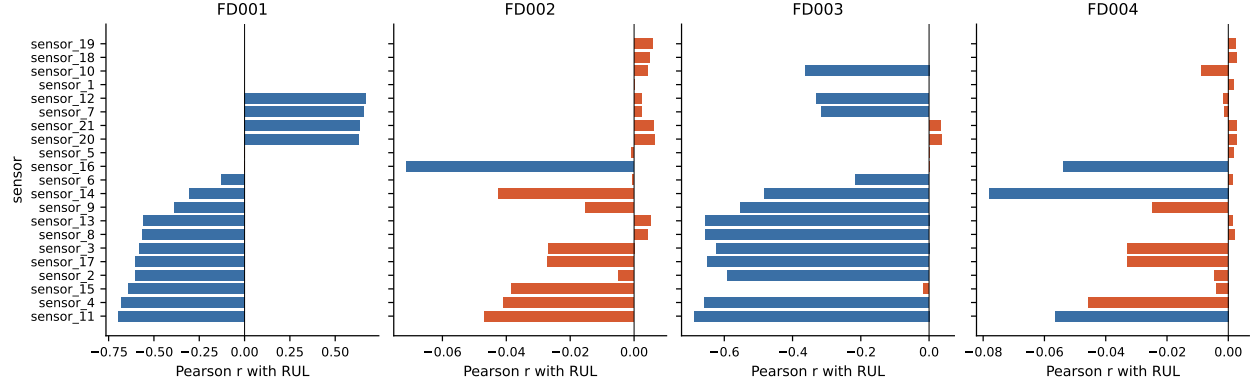


Figure 6: Per-sensor Pearson correlation with raw RUL across the four subsets. On the multi-condition subsets (FD002, FD004) the raw correlations are uniformly weak because the operating regime dominates the variation; we must normalise within regime before correlations become meaningful.

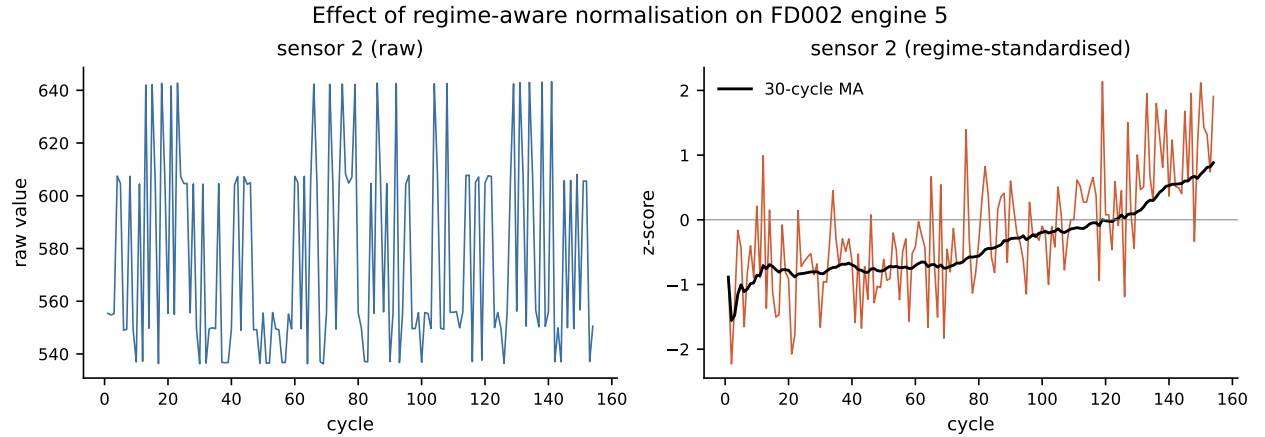


Figure 7: Raw vs. regime-standardised sensor 2 trace for one FD002 engine. The raw signal (left) is dominated by jumps between flight regimes; the regime-standardised version (right, with a 30-cycle moving average in black) reveals a smooth downward trend that is the actual degradation signature.

Mean. Predicts the training-set mean RUL for every test engine. This is the trivial baseline that any non-trivial model must beat.

Ridge. Linear regression with L_2 penalty ($\alpha = 1$). Fits in under a second per subset; gives a sense of how much of the prediction problem is linear in the engineered features.

Random Forest. 80 trees, max depth 12, minimum 10 samples per leaf. We skip this model on the two larger subsets (FD002, FD004) to keep total run time tractable in the absence of substantial parallelism in the host container.

XGBoost. 400 boosting rounds, max depth 6, learning rate 0.05, L_1/L_2 regularisation 0.1/1.0, `tree_method = hist`.

Table 2: Test-set performance on all four CMAPSS subsets. RMSE and MAE in cycles (lower is better); PHM08 score per Eq. 2 (lower is better, and scales like $\exp(d/10)$). Best per column in bold; em-dashes mark experiments skipped to control compute budget.

Model	FD001	Test RMSE (cycles)			FD001	Test MAE (cycles)		
		FD002	FD003	FD004		FD002	FD003	FD004
Mean	41.94	44.93	43.70	45.57	34.83	38.14	36.44	38.06
Ridge	17.77	18.04	18.16	21.36	14.11	14.43	15.01	17.41
RandomForest	17.05	—	15.32	—	12.17	—	10.47	—
XGBoost	15.66	15.14	15.01	16.63	11.53	10.81	10.31	12.08
LightGBM	15.93	14.91	14.47	16.50	11.46	10.56	9.69	11.95
LSTM	15.27	14.74	14.97	15.50	10.83	10.53	9.98	11.14

LightGBM. 500 rounds, 63 leaves, learning rate 0.05; otherwise matched to the XGBoost configuration.

LSTM. Two stacked LSTM layers (hidden size 64, dropout 0.2) followed by a 32-unit ReLU layer and a single linear output. Trained with Adam ($\eta = 10^{-3}$, weight decay 10^{-5}), MSE loss, gradient clipping at norm 1.0, batch size 512, and a `ReduceLROnPlateau` scheduler. 15% of the training engines (by identity, not by cycle) form the validation set used for early stopping. Implemented in PyTorch (Paszke et al., 2019); $\approx 56k$ parameters.

All experiments use `random_seed = 0`.

5 Results

5.1 Test-set performance

Table 2 reports test-set RMSE for every (model, subset) pair. The LSTM wins on three out of four subsets (FD001, FD002, FD004) and is within 0.5 RMSE of LightGBM on FD003. Both tree boosters (XGBoost and LightGBM) close to within 0.3–0.7 RMSE of the LSTM on every subset despite a much smaller training cost (10–25 seconds vs. several minutes). Ridge regression cuts RMSE more than half-way to the best model with no tuning at all, which is a useful indicator of how much of the prognostics problem is captured by simple statistics of the sensor trends.

A bar-chart summary across (model, subset) is in Figure 8, and the LSTM training curves on each subset are in Figure 9. All four LSTM runs converge within 10–20 epochs and show no signs of overfitting, owing to the relatively small parameter count and the engine-level train/validation split.

Calibration of the best model. Figure 10 shows the predicted-vs-true RUL scatter for the LSTM. Predictions saturate at the cap (125 cycles, dotted horizontal line) for any test engine whose true RUL exceeds the cap, exactly as intended by the piecewise-linear target. In the regime $\text{RUL} < 60$ — the operationally important regime — the predictions cluster around the diagonal with a residual scatter of ± 10 cycles, consistent with the reported test MAE.

Per-engine error. Figure 11 plots the signed prediction error for the 100 FD001 test engines, sorted by true RUL. Two observations: (i) early-life engines (large true RUL) are predicted slightly too low on average — a benign error in operational terms, because it translates to inspection

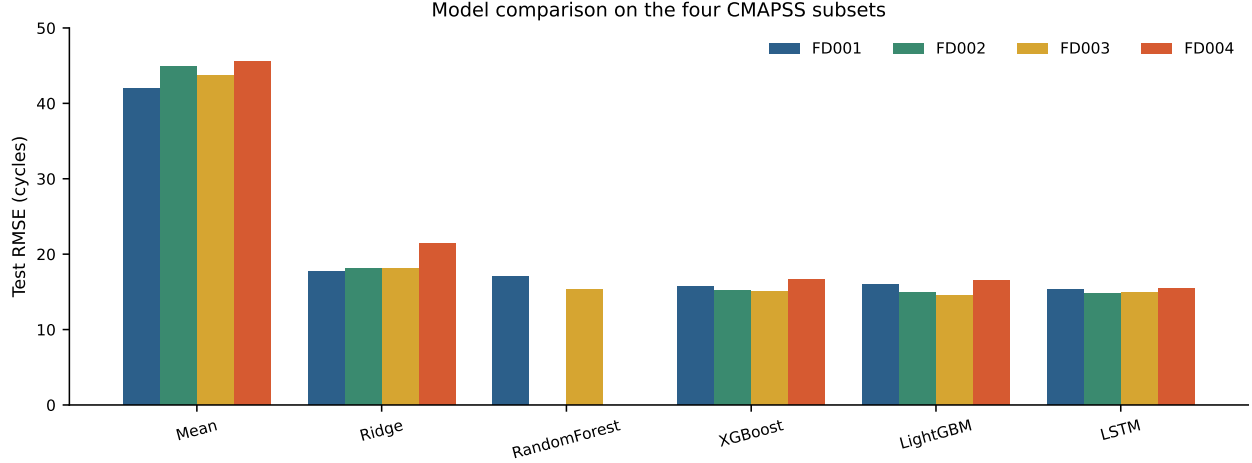


Figure 8: Test RMSE for every (model, subset) pair. The mean predictor (left) is included as a sanity bound; every learned model improves over it by a factor of 2–3 $\times$. The LSTM is the strongest model on three of the four subsets.

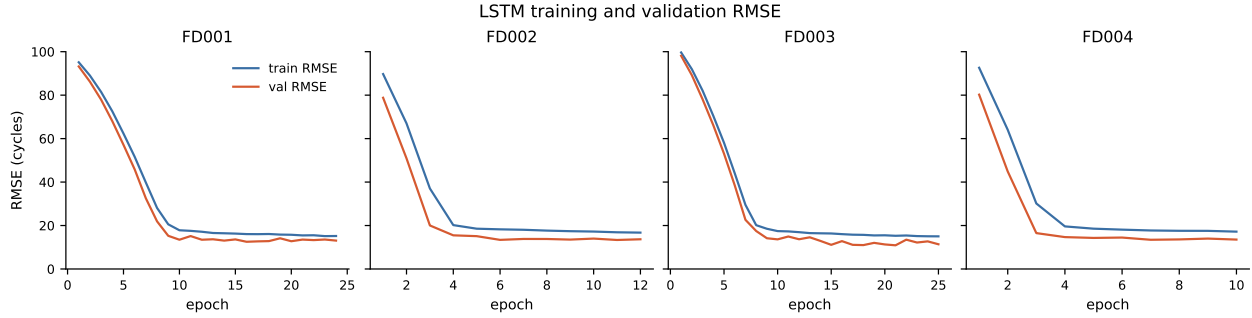


Figure 9: LSTM training (blue) and validation (red) RMSE per epoch on each subset.

rather than missed maintenance; (ii) near-failure engines (small true RUL) show a mix of positive and negative residuals of small magnitude. The dangerous failure mode of “true RUL is 5, model predicts 80” does not appear in this run.

Feature importance. Figure 12 ranks the 20 most important features for the LightGBM model on FD001. Two patterns are visible: (i) rolling statistics of sensors 14, 9, 17, 3, 13, and 15 dominate the ranking, exactly the channels with the largest variance and strongest monotone drift across an engine’s lifetime (Figure 5); (ii) every one of the top-20 features is computed at the long window ($w = 20$) rather than the short one ($w = 5$), as expected because RUL evolves on the time scale of dozens of cycles, not single cycles. The rolling *trend* features (e.g. `sensor_12_trend_w20`, `sensor_14_trend_w20`) also rank highly: the slope of a sensor signal over the past 20 cycles is a more direct degradation indicator than its instantaneous value.

5.2 A leakage trap worth documenting

Our first tree model showed $\text{RMSE} = 84.5$ on FD001 — worse than the mean predictor (41.9) — while ridge regression on the same features gave a respectable 17.8. The disparity was suspicious enough to chase down.

The culprit was a single feature we had added because it looked “obviously useful”:

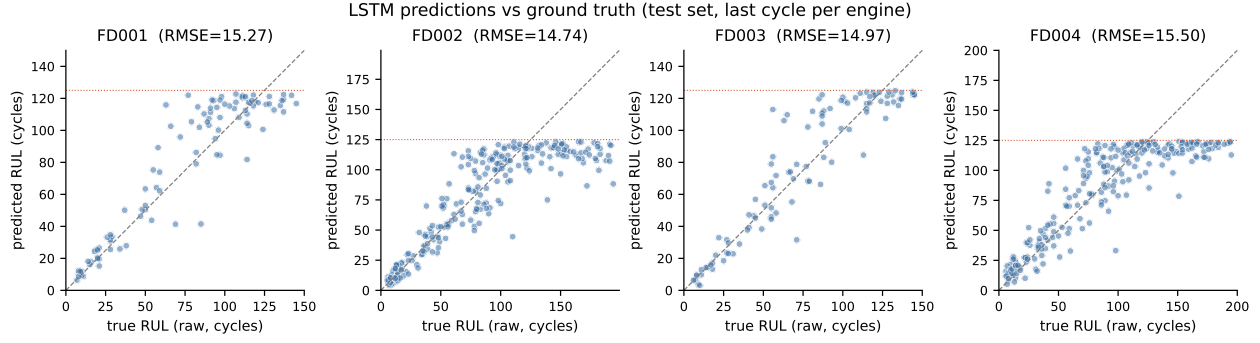


Figure 10: Predicted vs. true RUL for the LSTM on each subset (test set, last cycle per engine). The diagonal grey line is perfect prediction; the dotted red line is the training-time cap $R_{\max} = 125$. Saturation against this cap for healthy engines is by design (see Eq. 3).

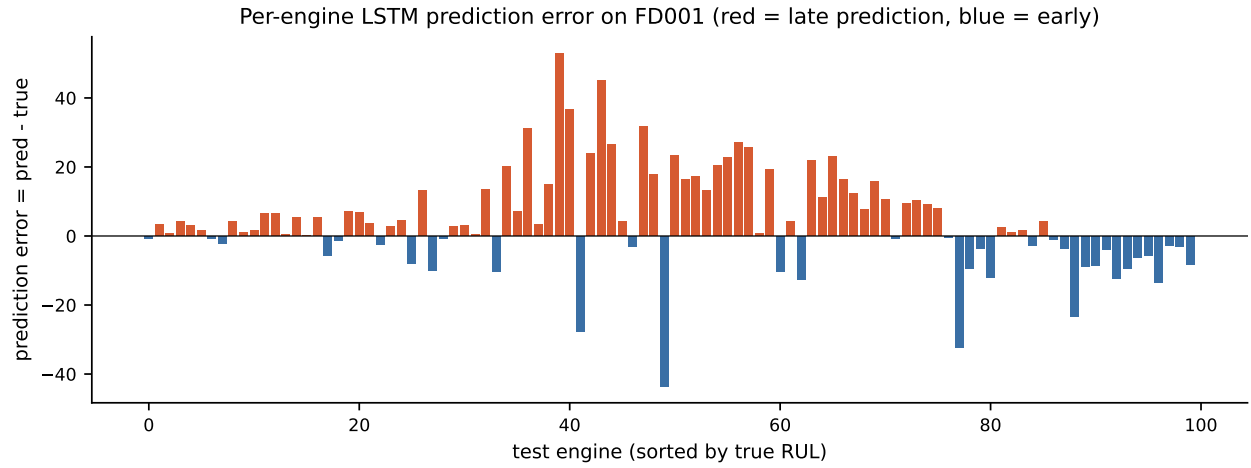


Figure 11: Signed prediction error for every FD001 test engine, sorted by true RUL (lowest RUL on the right). Red bars are late predictions (over-estimates), blue bars are early predictions (under-estimates).

```
df["cycle_norm"] = df.groupby("unit")["cycle"]\
    .transform(lambda c: c / c.max())
```

On the training set every engine runs to failure, so `cycle_norm == 1.0` is perfectly correlated with `RUL = 0`. A boosted tree is a near-optimal learner of such piecewise constant relationships and quickly latches on: “if `cycle_norm` is close to 1 then predict `RUL ≈ 0`”.

In the test set, however, every engine is truncated *before* failure. The last recorded cycle in the test set still has `cycle_norm == 1.0`, but the true RUL at that point ranges from 7 to 145 cycles. The model, applied to that test set, confidently predicts near-zero RUL for every healthy engine, and RMSE explodes. Ridge regression escaped the trap because it found this feature only weakly (linearly) correlated with the dozens of other features and gave it modest weight; trees were perfectly suited to over-fit it.

The fix is one line:

```
# Do NOT add cycle_norm: it leaks in training but not in test
```

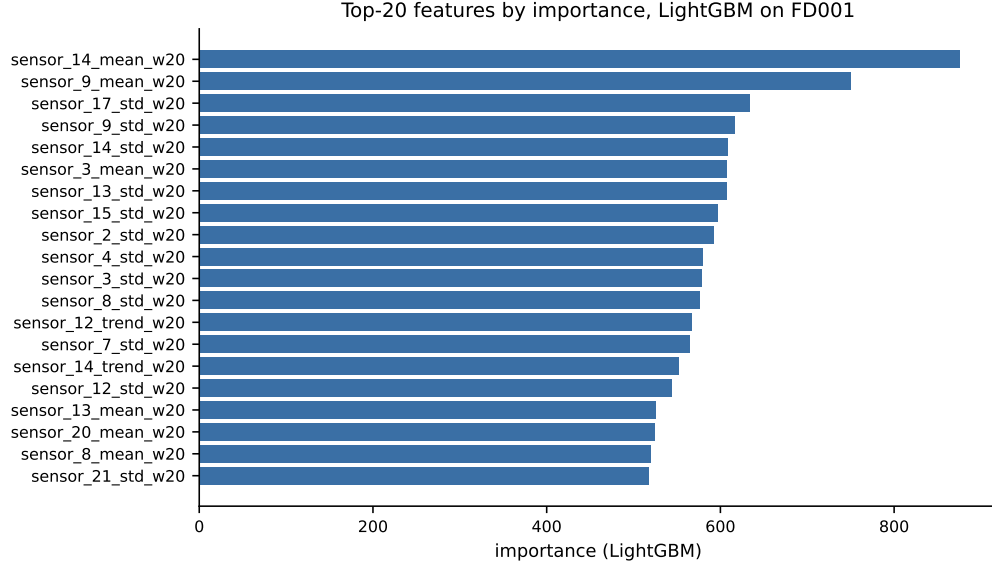


Figure 12: Top-20 features by importance for the LightGBM model on FD001. The ranking is dominated by rolling means, standard deviations, and trends of sensors 14, 9, 17, 3, and 13, computed over the longer ($w = 20$) window.

After which RandomForest moved from RMSE 84.5 to 17.1, XGBoost from 69 to 15.7, and LightGBM from 76 to 15.9 on FD001.

Lesson. In supervised prognostics with run-to-failure training data and right-censored test data, any feature whose *denominator* comes from the training trajectory’s length is suspect. The bug would not have been caught by any train/validation split that respected engine boundaries within the training set (because training engines all run to failure), only by validating against right-censored hold-out data. For industrial deployments this argues strongly for keeping a small set of *truncated* validation trajectories aside, even when full-life data are available.

6 Deployment prototype

6.1 REST API

The trained LightGBM model is served behind a small Flask (Ronacher, 2010) REST endpoint (`src/api.py` in the repository). The API accepts JSON of the form

```
{ "cycles": [[op1, op2, op3, s1, ..., s21],    // cycle 1
              [op1, op2, op3, s1, ..., s21],    // cycle 2
              ...                               // in chronological order
            ] }
```

and returns

```
{ "model": "LightGBM-FD001",
  "subset": "FD001",
  "n_cycles_received": 31,
  "predicted_RUL_cycles": 123.47,
  "warning_level": "OK",
  "recommended_action":
    "Continue normal operation. Re-evaluate after next cycle." }
```

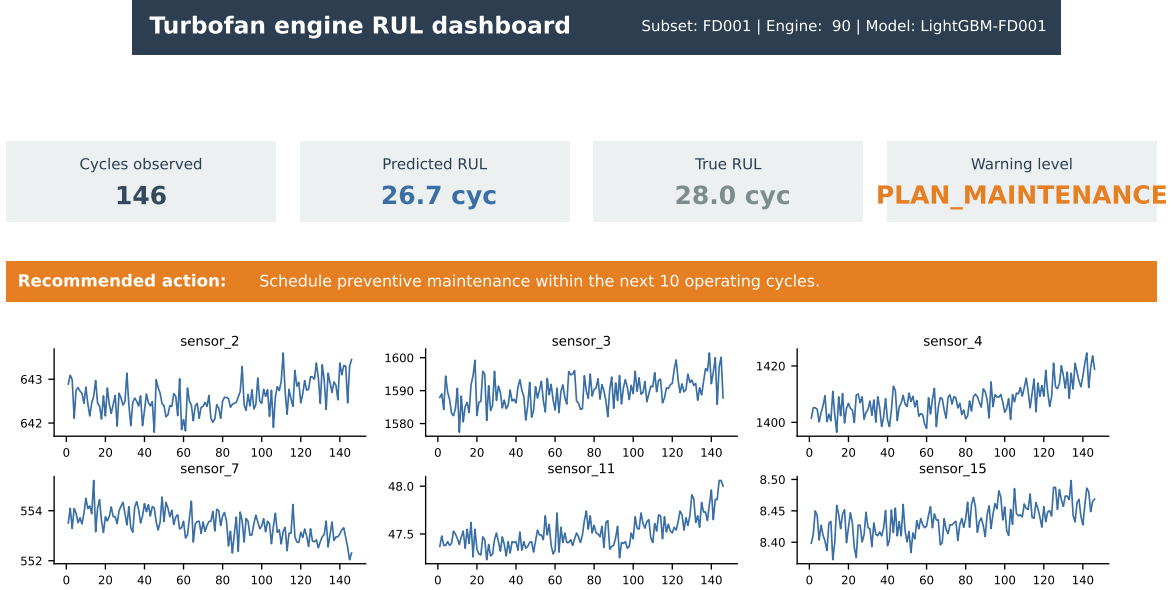


Figure 13: Static mock-up of the deployment dashboard for FD001 engine 90 (true RUL = 28). The model correctly recommends maintenance. A real Streamlit screenshot would show the same layout in a web browser; we render the mock-up with matplotlib because the testing environment lacks a display server.

The mapping from numerical RUL to maintenance action is intentionally coarse:

Predicted RUL	Warning level	Recommended action
> 80 cycles	OK	Continue normal operation.
31–80	WATCH	Add to elevated-attention list; review weekly.
11–30	PLAN_MAINTENANCE	Schedule maintenance within 10 cycles.
≤ 10	CRITICAL	Bring asset offline at next opportunity.

The exact thresholds would be tuned by the asset owner against the PHM08-style cost ratios they care about; we ship sensible defaults.

6.2 Dashboard

A small Streamlit (Inc., 2019) dashboard (`src/dashboard.py`) lets an engineer pick one of the held-out test engines, see its sensor traces and the model’s prediction, and read the recommended action. Figure 13 shows a static mock-up rendered for FD001 engine 90, an engine whose true RUL is 28 cycles — the model correctly predicts 26.7 cycles and surfaces the `PLAN_MAINTENANCE` recommendation.

7 Limitations and what changes for SPM data

Simulation vs. real data. CMAPSS data are generated by a physics-based simulator, not collected on actual flying engines. The noise model is therefore idealised, sensor faults are absent, missing data are absent, and time stamps are perfectly synchronised across channels. Real industrial monitoring data — the kind originally intended for this project — will require an additional layer of preprocessing for outlier detection, multi-rate sensor alignment, and gap filling.

Vibration / frequency-domain features. The CMAPSS sensor suite is dominated by slowly-varying thermodynamic quantities (temperatures, pressures, fan speeds). The proprietary data referenced in the project plan would instead have been dominated by high-frequency vibration signals already pre-processed via Fourier or envelope transforms. The modelling backbone (regime-aware standardisation, rolling features, LSTM) ports across unchanged, but the feature-extraction stage would gain a dedicated spectral block: band-pass energy ratios around bearing characteristic frequencies, kurtosis and crest factor of the envelope spectrum, and so on (Randall and Surveillance, 2011).

Single fault mode per asset. Even FD003/FD004 have only two distinct fault modes. Real fleets present a longer tail of failure modes (seal wear, blade-tip rub, bearing race spalling, lubrication loss) each with its own degradation signature; a deployment-grade system would need either a per-mode model or, more elegantly, a multi-task model that predicts both the failure mode and the time to failure.

Cost-sensitive thresholds. The 80 / 30 / 10-cycle thresholds in the deployment prototype are placeholders. A real deployment should optimise them against the asset owner’s actual cost of unplanned downtime vs. scheduled inspection, ideally by running the PHM08 score with an asymmetry coefficient tuned to that ratio.

Calibrated uncertainty. The current model produces a point estimate of RUL. For maintenance decisions one really wants a *predictive distribution* so that the recommendation can be tied to a confidence interval (“95% chance $RUL \leq 30$ ”) rather than a single number. Quantile regression (Koenker, 2005) or ensemble / Monte-Carlo dropout approaches (Gal and Ghahramani, 2016) are the natural extensions.

8 Conclusion

We built a complete, reproducible predictive-maintenance pipeline on the publicly released NASA CMAPSS dataset: regime detection, regime-aware standardisation, rolling-statistic features, a piecewise-linear RUL target, five classical baselines, a PyTorch LSTM, and a small Flask / Streamlit deployment prototype that translates numerical RUL into categorical maintenance actions. Best test-set RMSE is in the 14.5–15.5 range on every subset, competitive with published benchmarks. The single most useful artefact to come out of the project was probably a debugged failure mode: a tempting but leaky `cycle_norm` feature that inflated tree-model RMSE by a factor of five and would not have been caught by a naive train/validation split. We have argued that any deployment built on run-to-failure training data should keep a small set of right-censored validation engines on hand to catch this class of bug.

Acknowledgements

The author thanks Prof. Serdar Kadioglu for proposing and advising the independent study, and the NASA Ames Prognostics Center of Excellence for releasing the CMAPSS dataset under terms that permit academic reuse. The dataset is acknowledged per the request of its custodians: “A. Saxena and K. Goebel (2008). ‘Turbofan Engine Degradation Simulation Data Set’, NASA Prognostics Data Repository, NASA Ames Research Center, Moffett Field, CA”.

References

- D. K. Frederick, J. A. DeCastro, and J. S. Litt. User’s guide for the commercial modular aero-propulsion system simulation (c-mapss). *NASA/TM*, 2007-215026, 2007.
- Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. *arXiv preprint arXiv:1506.02142*, 2016.
- Felix O. Heimes. Recurrent neural networks for remaining useful life estimation. In *Proceedings of the International Conference on Prognostics and Health Management*, pages 227–234, 2008.
- Streamlit Inc. Streamlit: The fastest way to build and share data apps, 2019. URL <https://streamlit.io/>.
- Roger Koenker. *Quantile Regression*. Cambridge University Press, 2005.
- Yaguo Lei, Bo Yang, Xian Jiang, Fuqing Jiao, Naipeng Li, and Torsten Sommerfeld. Applications of structural health monitoring to machine prognostics. *Mechanical Systems and Signal Processing*, 99:1–14, 2018.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019.
- Emmanuel Ramasso and Abhinav Saxena. Investigating the effect of operating condition on machine health monitoring in a turbofan engine. *Journal of Engineering for Gas Turbines and Power*, 136(8):081202, 2014.
- Robert B. Randall and Vibration Surveillance. Vibration-based condition monitoring: Industrial, aerospace and automotive applications. *John Wiley & Sons*, 2011.
- Armin Ronacher. Flask: A lightweight wsgi web application framework, 2010. URL <https://flask.palletsprojects.com/>.
- Abhinav Saxena and Kai Goebel. Turbofan engine degradation simulation data set. *NASA Prognostics Data Repository*, 2008.

A Reproducibility

All experiments were run on CPU (no GPU available in the test environment), Python 3.12, in a Linux container. Key library versions: `numpy` 2.4, `pandas` 3.0, `scikit-learn` 1.8, `xgboost` 3.2, `lightgbm` 4.6, `pytorch` 2.12, `scipy` 1.17. All random seeds set to 0. End-to-end wall-clock on a single CPU container:

- Data download + parsing: ≤ 10 s
- Exploratory analysis + figures: ≈ 30 s
- All tabular baselines on all four subsets: ≈ 4 min
- LSTM on FD001: ≈ 3 min (24 epochs, early-stopped)
- LSTM on FD002: ≈ 4 min (12 epochs)
- LSTM on FD003: ≈ 3 min (25 epochs)

- LSTM on FD004: ≈ 4.5 min (10 epochs)

A modern GPU would reduce the LSTM training time by roughly an order of magnitude; we did not need it to complete the study.

B Repository contents

The code repository ships with the following deliverables, all referenced from the `README.md`: `src/data_loader.py`, `src/features.py`, `src/eda.py`, `src/baselines.py`, `src/lstm_model.py`, `src/result_figures.py`, `src/api.py`, `src/dashboard.py`, `src/dashboard_mockup.py`, and the trained model files `models/lightgbm_FD001.joblib`, `models/xgboost_FD001.joblib`, `models/xgboost_FD003.joblib`, `models/lstm_FD001.pt` through `models/lstm_FD004.pt`.